

《神经网络与深度学习》

第七章-网络优化与正则化

人工智能学院

杨溪

优化

经验风险最小

正则化

降低模型复杂度
提高泛化能力



➤ 网络优化

➤ 神经网络优化的难点

➤ 优化算法

➤ 优化算法改进

➤ 动态学习率

➤ 梯度估计修正

➤ 其他优化

➤ 参数初始化

➤ 数据预处理

➤ 逐层规范化

➤ 超参数优化

➤ 网络正则化

➤ 早停法

➤ l_1 和 l_2 正则化

➤ 暂退法 (Dropout)

➤ 数据增强

网络优化

◆ 结构差异大

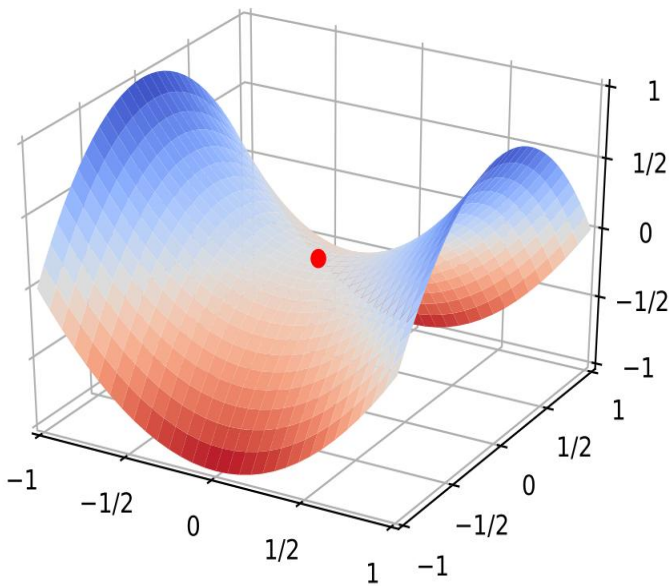
- ◆ 没有通用的优化算法
- ◆ 超参数多

◆ 非凸优化问题

- ◆ 参数初始化
- ◆ 逃离局部最优或鞍点

◆ 梯度消失（爆炸）问题

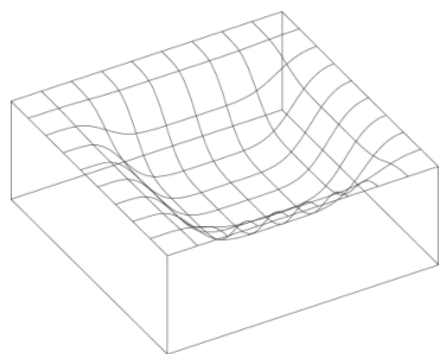
- ◆ 在高维空间中，非凸优化的难点并不在于如何逃离局部最优
点，而是如何逃离鞍点（梯度为 0 的点）



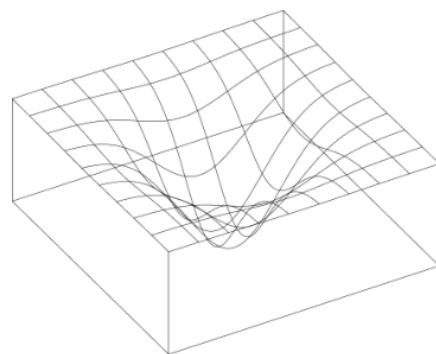
局部最小值（每一维度上都是最低点）
的概率非常低。

◆ 平坦最小值 (Flat Minima)

- ◆ 一个平坦最小值的邻域内，所有点对应的训练损失都比较接近
- ◆ 大部分的局部最小解是等价的
- ◆ 局部最小解对应的训练损失都可能非常接近于全局最小解对应的训练损失



(a) 平坦最小值



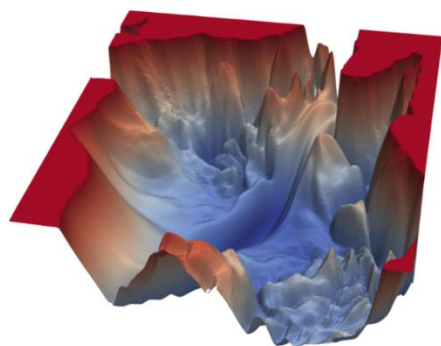
(b) 尖锐最小值

局部最优 ✓

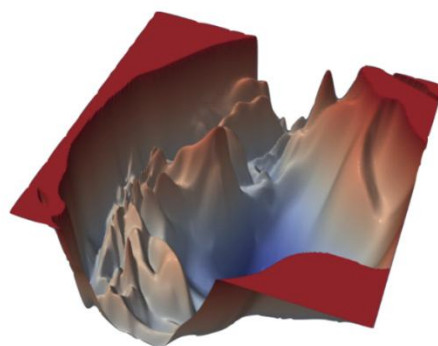
全局最优

优化地形 (Optimization Landscape) :
在高维空间中损失函数的曲面形状

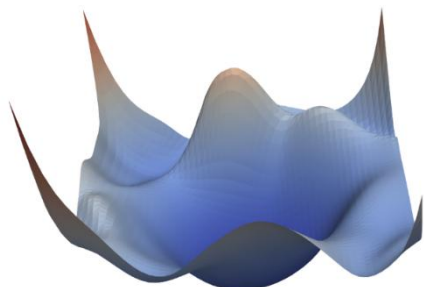
VGG-56



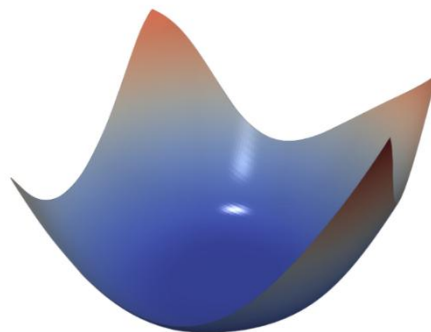
VGG-110



Renset-56



Densenet-121



skip connections !

- ◆ 更有效的优化算法来提高优化方法的效率和稳定性
 - ◆ 动态学习率调整
 - ◆ 梯度估计修正
- ◆ 更好的参数初始化方法、数据预处理方法来提高优化效率
- ◆ 修改网络结构来得到更好的优化地形
 - ◆ 好的优化地形通常比较平滑
 - ◆ 使用 ReLU 激活函数、残差连接、逐层归一化等
- ◆ 使用更好的超参数优化方法

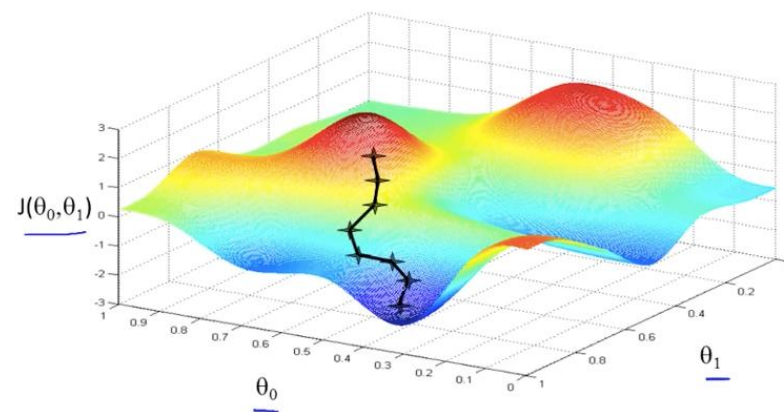
优化算法改进

◆ 随机梯度下降可降低每次迭代时的计算代价。在随机梯度下降的每次迭代中，我们对数据样本随机均匀采样一个索引，并计算梯度以更新。

算法 2.1 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

```
1 随机初始化  $\theta$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机排序;  
4   for  $n = 1 \dots N$  do  
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6      $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; \mathbf{x}^{(n)}, y^{(n)})}{\partial \theta}$ ; // 更新参数  
7   end  
8 until 模型  $f(\mathbf{x}; \theta)$  在验证集  $\mathcal{V}$  上的错误率不再下降;  
输出:  $\theta$ 
```



◆ 选取 K 个训练样本，计算偏导数

$$g_t(\theta) = \frac{1}{K} \sum_{(x,y) \in \mathcal{S}_t} \frac{\partial \mathcal{L}(y, f(x; \theta))}{\partial \theta}$$

◆ 定义梯度

$$g_t \triangleq g_t(\theta_{t-1})$$

◆ 更新参数

$$\theta_t \leftarrow \theta_{t-1} - \alpha g_t$$

几个关键因素：

- 小批量样本数量 K
- 梯度
- 学习率 α

◆ 批量大小的影响

◆ 批量大小 **不影响随机梯度的期望**，但是会 **影响随机梯度的方差**。批量越大，随机梯度的方差越小，引入的噪声也越小，训练也越稳定，因此可以设置较大的学习率。

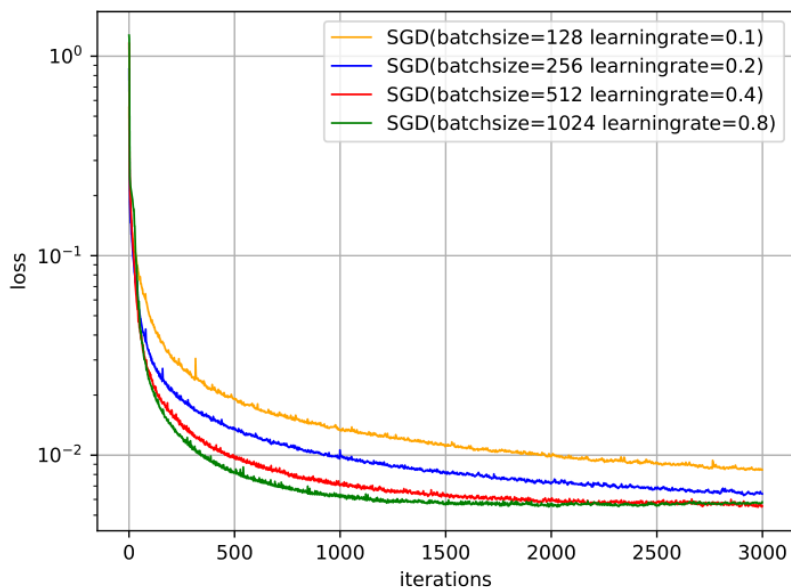
◆ 而批量较小时，需要设置较小的学习率，否则模型会不收敛。

◆ 批量大小和学习率的关系

◆ 线性缩放规则

当批量大小增加 m 倍时，学习率也增加 m 倍

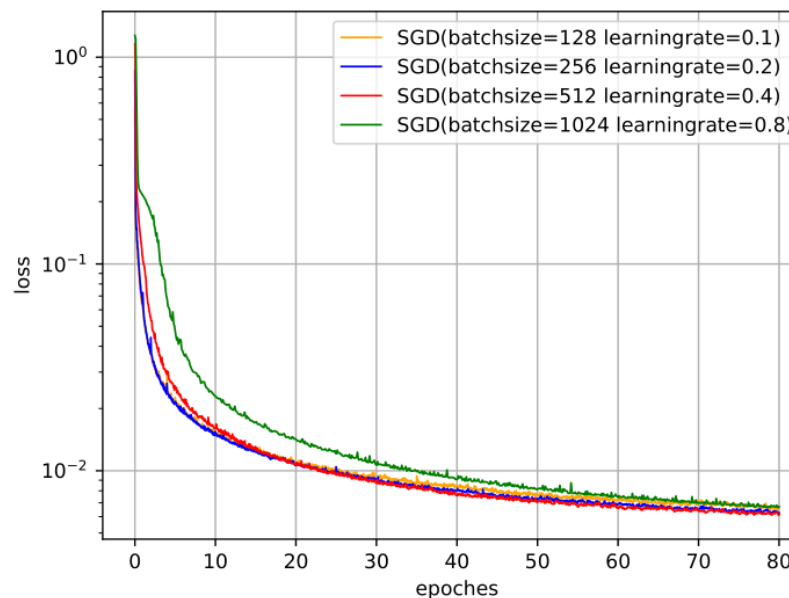
◆ 批量大小的影响



(a) 按 Iteration 的损失变化

batchsize大 下降快

样本多用大batchsize
样本少用小batchsize



(b) 按 Epoch 的损失变化

batchsize小 下降快

大batchsize效率高
小batchsize模型泛化性更好

4 种批量大小
对应的学习率
设置不同，因
此并不是严格
对比。

◆ 学习率

◆ 学习率衰减

◆ Adagrad

◆ Adadelta

◆ RMSprop

◆ 梯度

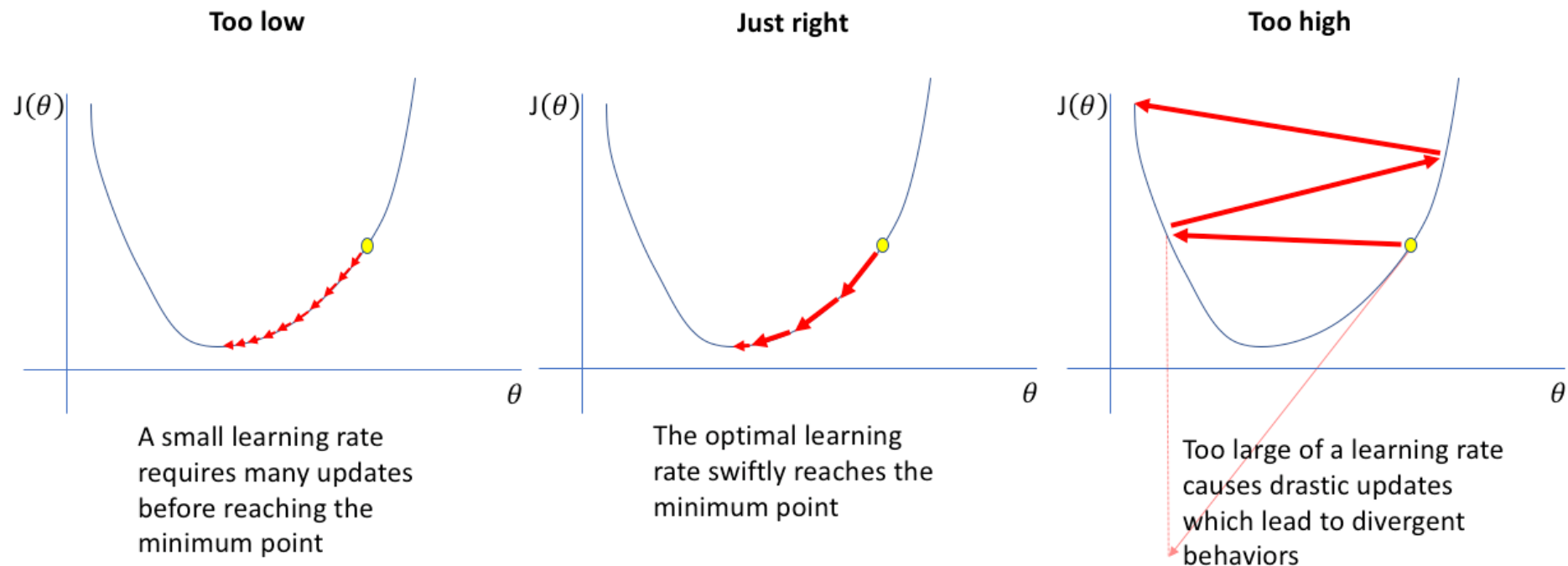
◆ Momentum

◆ 计算负梯度的“加权移动平均”作为参数的更新方向

◆ Nesterov accelerated gradient

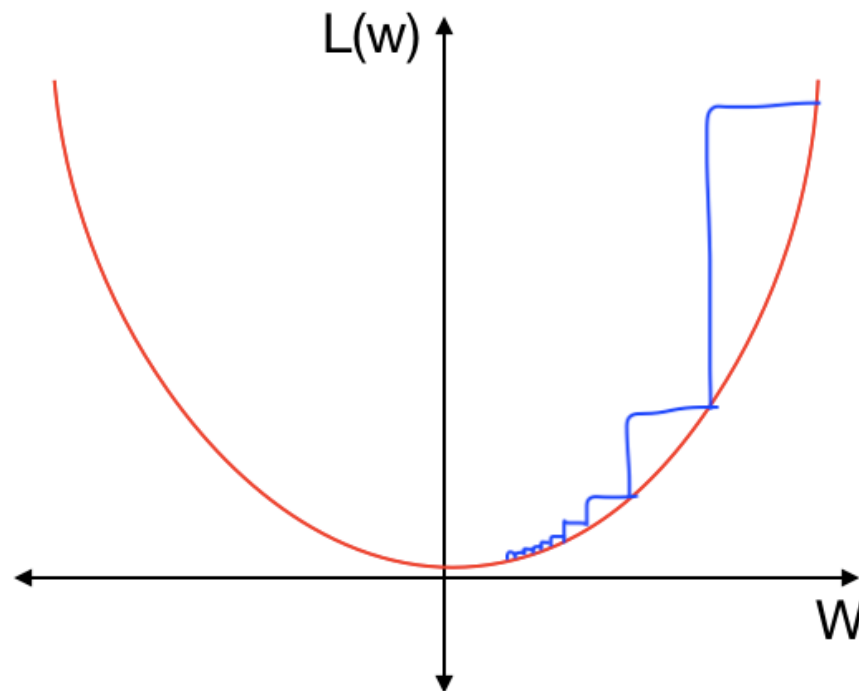
◆ 梯度截断

◆ 学习率的影响

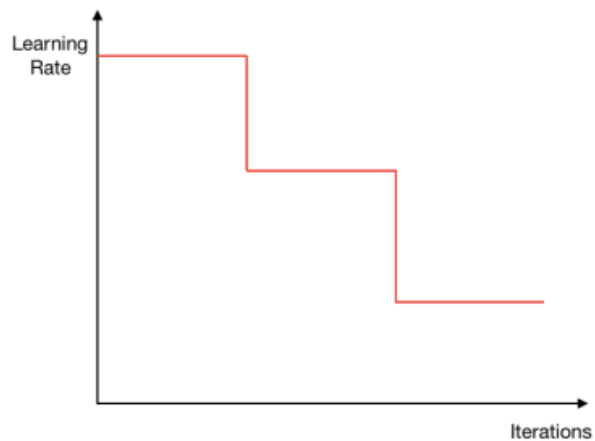


◆ 学习率衰减

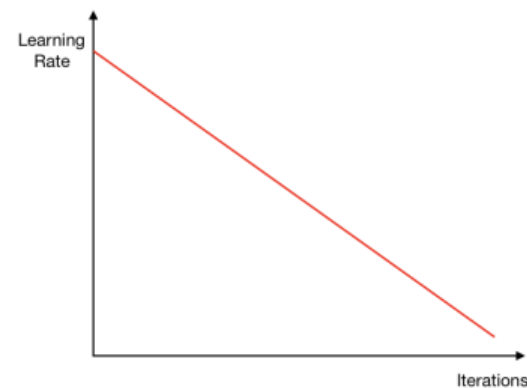
◆ 初始化学学习率为 α_0 ，在第 t 次迭代时的学习率衰减到 α_t



◆ 学习率衰减



梯级衰减 (step decay)



线性衰减 (Linear Decay)

◆ 常见的学习率衰减方法有以下几种：

◆ 分段常数衰减

◆ 逆时衰减

$$\alpha_t = \alpha_0 \frac{1}{1 + \beta \times t}$$

◆ 指数衰减

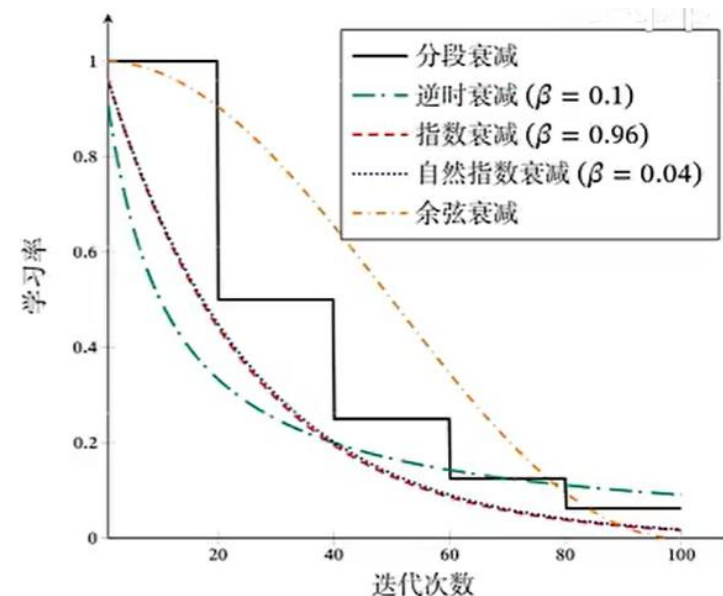
$$\alpha_t = \alpha_0 \beta^t$$

◆ 自然指数衰减

$$\alpha_t = \alpha_0 \exp(-\beta \times t)$$

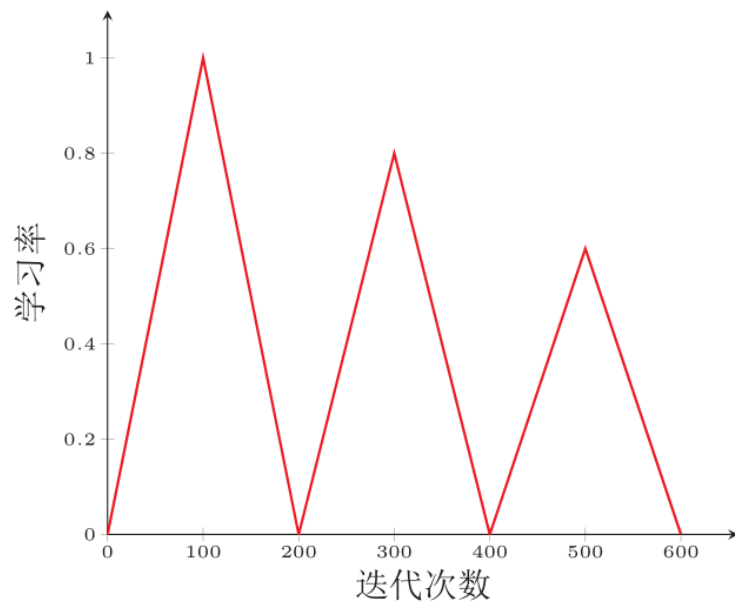
◆ 余弦衰减

$$\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos\left(\frac{t\pi}{T}\right) \right)$$

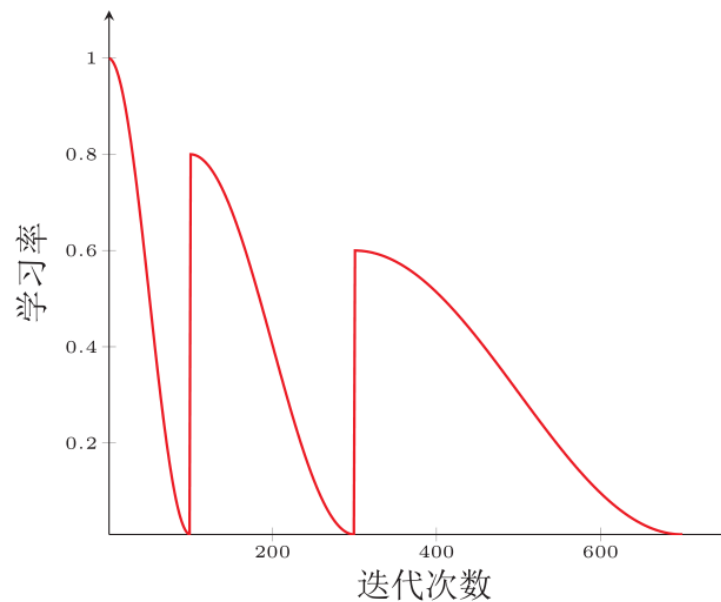


α_0 初始学习率
 β 衰减率
 t 时间

◆ 周期性学习率调整



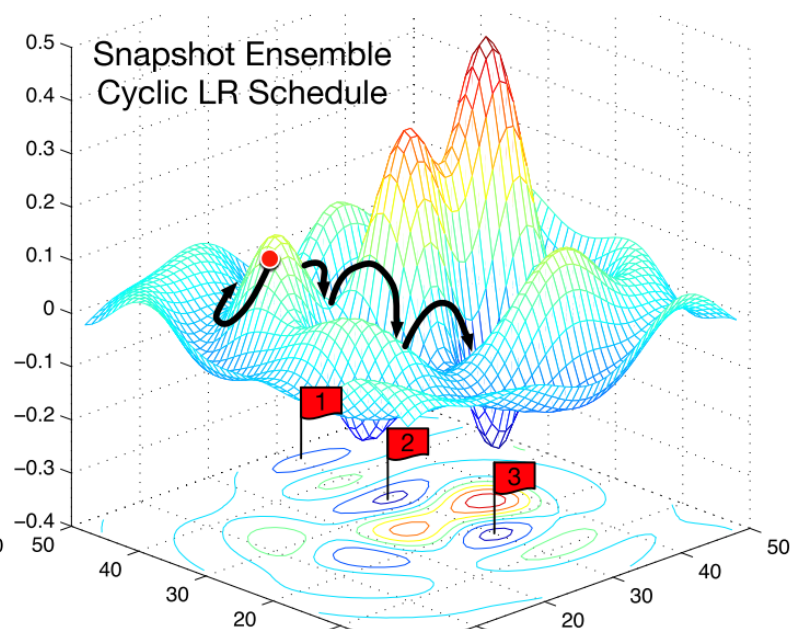
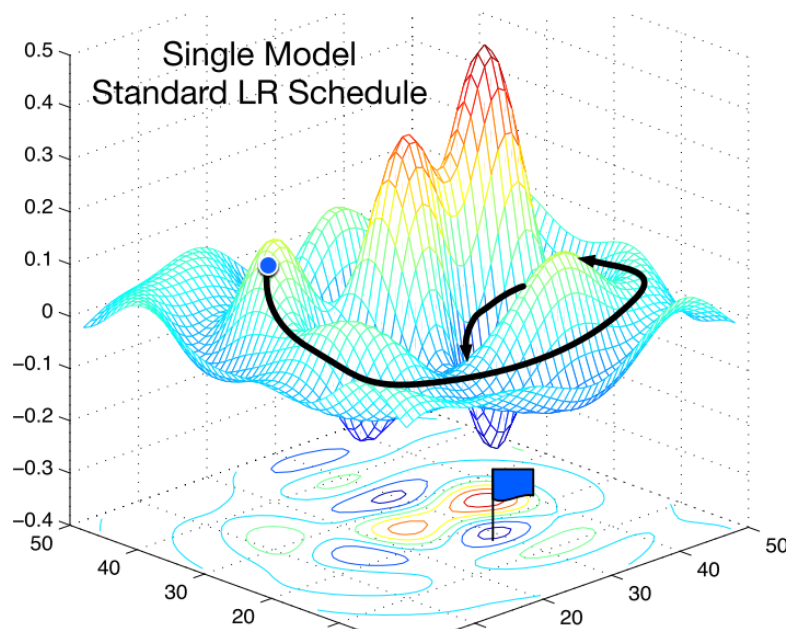
(a) 三角循环学习率



(b) 带热重启的余弦衰减

跳出局部最优
跳出尖锐局部最优

◆ 周期性学习率调整



◆ 增加批量大小

◆ Don't Decay the Learning Rate, Increase the Batch Size

◆ ICLR 2018

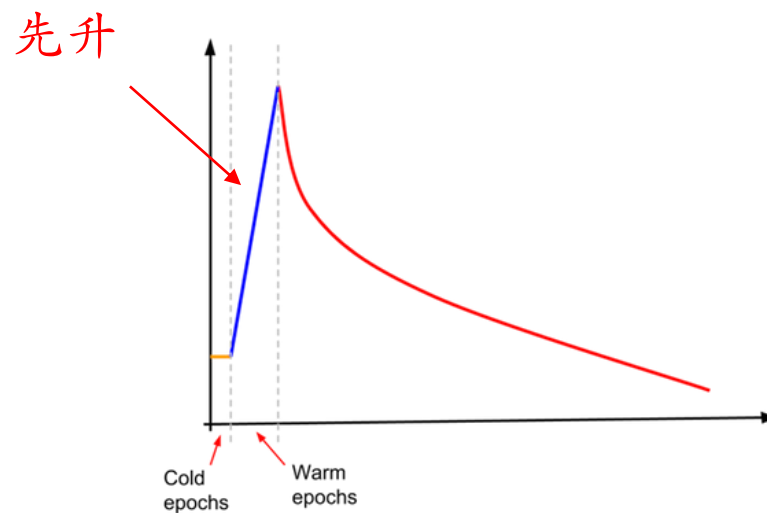
◆ <https://openreview.net/pdf?id=B1Yy1BxCZ>

◆ 学习率预热 (Warmup)

◆ Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour

◆ Arxiv

◆ <https://arxiv.org/abs/1706.02677>



◆ 自适应学习率（根据梯度动态改变学习率）

◆ AdaGrad算法：根据参数的历史梯度信息来动态地调整学习率。

计算第 t 次迭代时每个参数梯度平方的累计值 $G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}$

参数更新差值 $\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$

参数更新 $\theta_{t+1} = \theta_t + \Delta\theta_t$

α : 初始学习率

G_t : 参数 w 的历史梯度平方和

ϵ : 一个很小的常数, 如 10^{-8} ,
- 用于避免除以零的情况

随着累计, 不断缩小。

◆ 自适应学习率

◆ RMSprop 算法:旨在解决AdaGrad算法中学习率过早衰减的问题。

计算第 t 次迭代时每个参数梯度平方的移动平均

RMSprop使用指数加权移动平均来计算累积梯度平方,其中使用指数衰减 β 来降低旧梯度的影响

参数更新差值
$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

$$\begin{aligned} G_t &= \beta G_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t \\ &= (1 - \beta) \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_\tau \odot \mathbf{g}_\tau \end{aligned}$$

可以变小也可以变大。

◆ 自适应学习率

◆ Adadelta算法:与RMSprop相似,但它利用参数更新差值 $\Delta\theta_t$ 来调整学习率。

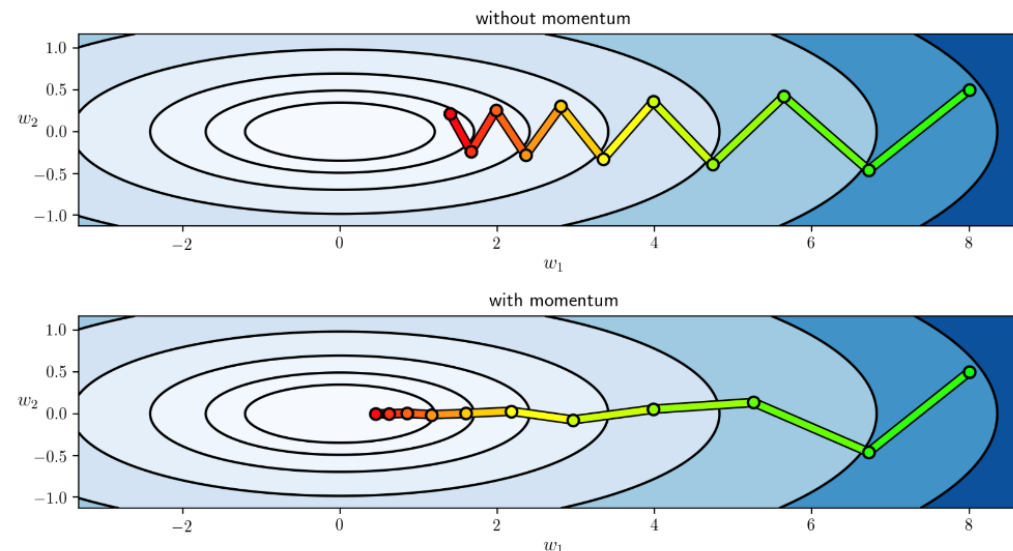
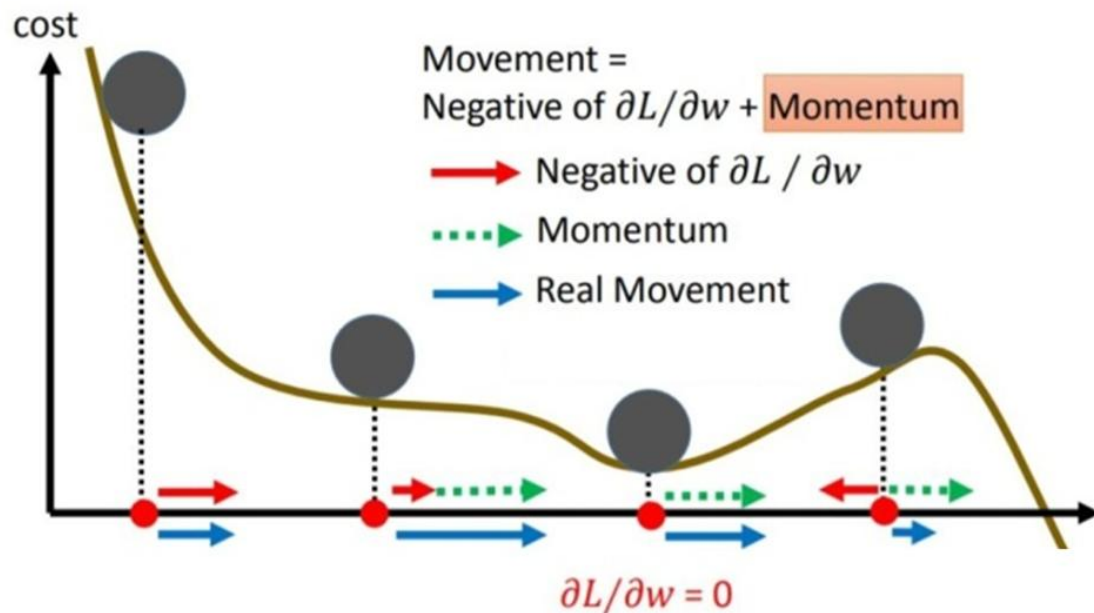
$$\Delta X_{t-1}^2 = \beta_1 \Delta X_{t-2}^2 + (1 - \beta_1) \underline{\Delta\theta_{t-1} \odot \Delta\theta_{t-1}}$$

参数更新差值 $\Delta\theta_t = -\frac{\sqrt{\Delta X_{t-1}^2 + \epsilon}}{\sqrt{G_t + \epsilon}} \mathbf{g}_t$

在一定程度上平抑了学习率的波动

◆ 动量法 (Momentum) : 用之前积累动量来替代真正的梯度, 每次迭代的梯度可以看作是加速度 (近似看作二阶梯度)。在第 t 次迭代时, 计算负梯度的“加权移动平均”作为参数的更新方向

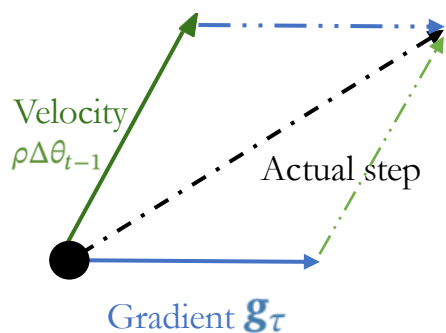
$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t = -\alpha \sum_{\tau=1}^t \rho^{t-\tau} \mathbf{g}_\tau, \quad \leftarrow \Delta\theta_t = -\alpha\mathbf{g}_t$$



◆ Nesterov 加速梯度

Momentum update:

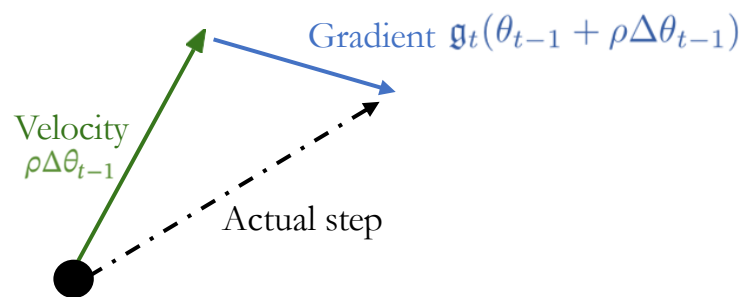
$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t = -\alpha \sum_{\tau=1}^t \rho^{t-\tau} \mathbf{g}_\tau$$



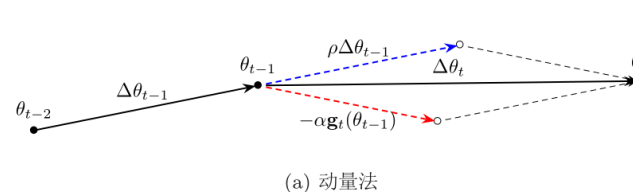
将当前点的梯度与速度相结合以获得用于更新权重的步长。

Nesterov Momentum:

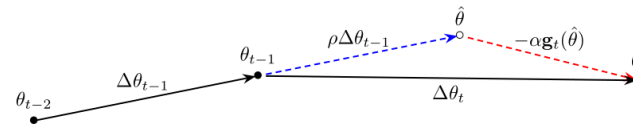
$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1})$$



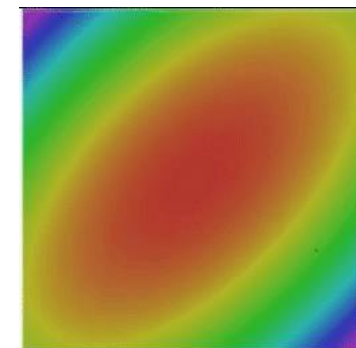
“向前看”到使用速度进行更新的程度；在那里计算梯度并将其与速度混合以获得实际的更新方向。



(a) 动量法



(b) Nesterov 加速梯度



- SGD
- SGD+ Momentum
- SGD+ Nesterov

◆ 梯度方向优化+自适应学习率

◆ Adam算法 $\approx$ 动量法+RMSprop

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) \mathbf{g}_t,$$

$$G_t = \beta_2 G_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t,$$

计算两个移动平均

通常取值:

$$\beta_1 = 0.9$$

$$\beta_2 = 0.99$$

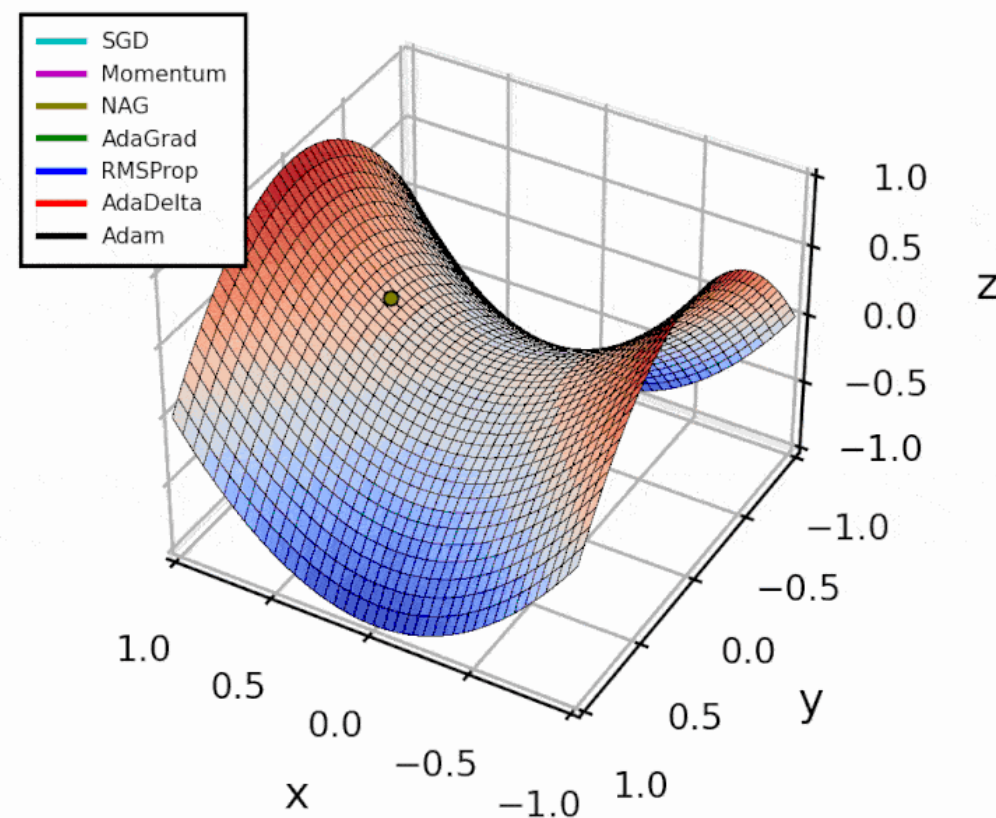
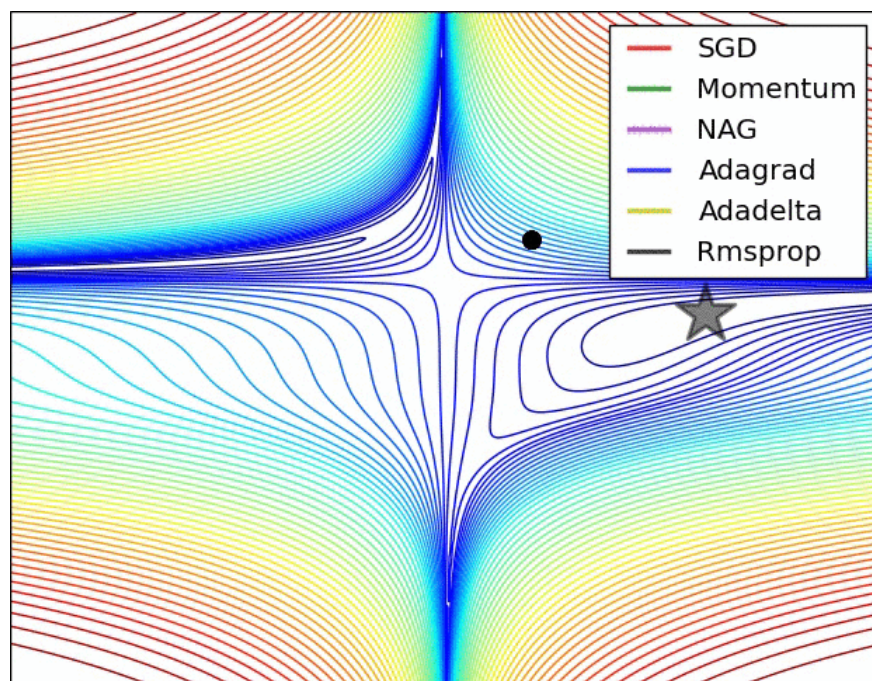
$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t},$$

$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}.$$

偏差修正

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{\hat{G}_t} + \epsilon} \hat{M}_t$$

参数更新



◆ 梯度截断

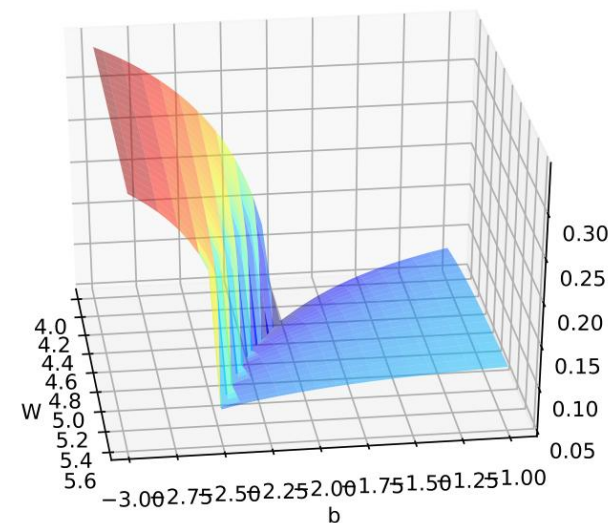
◆ 一种比较简单的启发式方法，把梯度的模限定在一个区间，当梯度的模小于或大于这个区间时就进行截断。

◆ 按值截断

$$\mathbf{g}_t = \max(\min(\mathbf{g}_t, b), a)$$

◆ 按模截断

$$\mathbf{g}_t = \frac{b}{\|\mathbf{g}_t\|} \mathbf{g}_t$$



NaN

◆ 大部分优化算法可以使用下面公式来统一描述概括

$$\Delta\theta_t = -\frac{\alpha_t}{\sqrt{G_t + \epsilon}}M_t,$$

α_t 为第t步的学习率

$$G_t = \psi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$\mathbf{g}_t$ 为第t步的梯度

$$M_t = \phi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam $\approx$ 动量法 + RMSprop

参数初始化

- ◆好的初始值会使得网络收敛到一个泛化能力高的局部最优解
- ◆参数不能初始化为0!
 - ◆对称权重问题
- ◆初始化方法
 - ◆预训练初始化
 - ◆随机初始化
 - ◆固定值初始化
 - ◆偏置 (Bias) 通常用 0 来初始化

◆ 随机初始化:

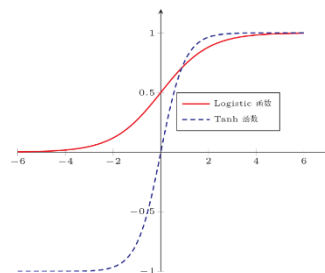
◆ Gaussian分布初始化

◆ Gaussian初始化方法是最简单的初始化方法，参数从一个固定均值和固定方差的Gaussian分布进行随机初始化。

◆ 均匀分布初始化

◆ 参数可以在区间 $[-r, r]$ 内采用均匀分布进行初始化。

取值?



◆ 基于方差缩放的参数初始化

◆ 初始化一个深度网络时，根据神经元的连接数量来自适应地调整初始化分布的方差

◆ Xavier 初始化和 He 初始化

初始化方法	激活函数	均匀分布 $[-r, r]$	高斯分布 $\mathcal{N}(0, \sigma^2)$
Xavier 初始化	Logistic	$r = 4\sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = 16 \times \frac{2}{M_{l-1}+M_l}$
Xavier 初始化	Tanh	$r = \sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = \frac{2}{M_{l-1}+M_l}$
He 初始化	ReLU	$r = \sqrt{\frac{6}{M_{l-1}}}$	$\sigma^2 = \frac{2}{M_{l-1}}$

$$a^{(l)} = f\left(\sum_{i=1}^{M_{l-1}} w_i^{(l)} a_i^{(l-1)}\right)$$

$$\mathbb{E}[a^{(l)}] = \mathbb{E}\left[\sum_{i=1}^{M_{l-1}} w_i^{(l)} a_i^{(l-1)}\right] = \sum_{i=1}^{M_{l-1}} \mathbb{E}[w_i^{(l)}] \mathbb{E}[a_i^{(l-1)}] = 0$$

$$\begin{aligned} \text{var}(a^{(l)}) &= \text{var}\left(\sum_{i=1}^{M_{l-1}} w_i^{(l)} a_i^{(l-1)}\right) \\ &= \sum_{i=1}^{M_{l-1}} \text{var}(w_i^{(l)}) \text{var}(a_i^{(l-1)}) \\ &= M_{l-1} \text{var}(w_i^{(l)}) \text{var}(a_i^{(l-1)}). \end{aligned}$$

$$\text{var}(w_i^{(l)}) = \frac{1}{M_{l-1}} \quad \text{var}(a_i^{(l-1)}) = \frac{2}{M_{l-1} + M_l}$$

$$r = \sqrt{3\sigma^2} = \sqrt{\frac{6}{M_{l-1} + M_l}}$$

◆ 范数保持性 (Norm-Preserving)

◆ 一个 M 层的等宽线性网络

$$y = W^{(L)} W^{(L-1)} \dots W^{(1)} x$$

◆ 为了避免梯度消失或梯度爆炸问题，我们希望误差项

$$\|\delta^{(l-1)}\|^2 = \|\delta^{(l)}\|^2 = \|(\mathbf{W}^{(l)})^\top \delta^{(l)}\|^2$$

$$W^{(l)} \cdot (W^{(l)})^T = I$$

◆ 正交初始化 (常用于RNN)

- 1) 用均值为 0、方差为 1 的高斯分布初始化一个矩阵；
- 2) 将这个矩阵用奇异值分解得到两个正交矩阵，并使用其中之一作为权重矩阵。

数据预处理

◆ 在缩放全部或部分特征后不影响深度学习算法的学习和预测

ID	长度 (cm)	宽度 (cm)	类别
1	4.2	5.9	橙子
2	6.4	7.0	橙子
3	8.3	5.5	柠檬
...			
12	5.9	8.9	橙子
13	8.8	9.3	橙子
14	10.2	6.7	柠檬



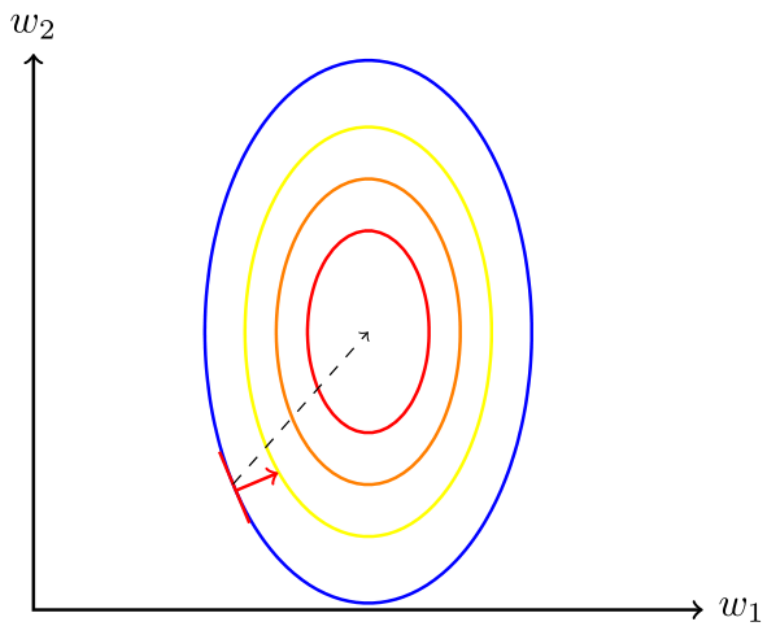
ID	长度 (cm)	宽度 (mm)	类别
1	4.2	59	橙子
2	6.4	70	橙子
3	8.3	55	柠檬
...			
12	5.9	89	橙子
13	8.8	93	橙子
14	10.2	67	柠檬

w_1

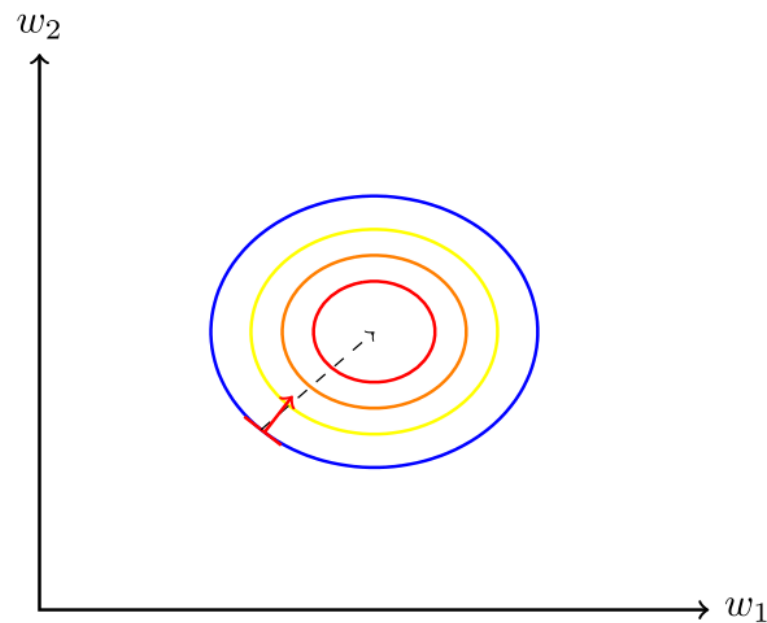
w_2

不同输入特征的尺度差异比较大时，对参数初始化和优化效率都有影响。

◆ 数据归一化对梯度的影响



(a) 未归一化数据的梯度



(b) 归一化数据的梯度

◆ 数据归一化

◆ 最小最大值归一化

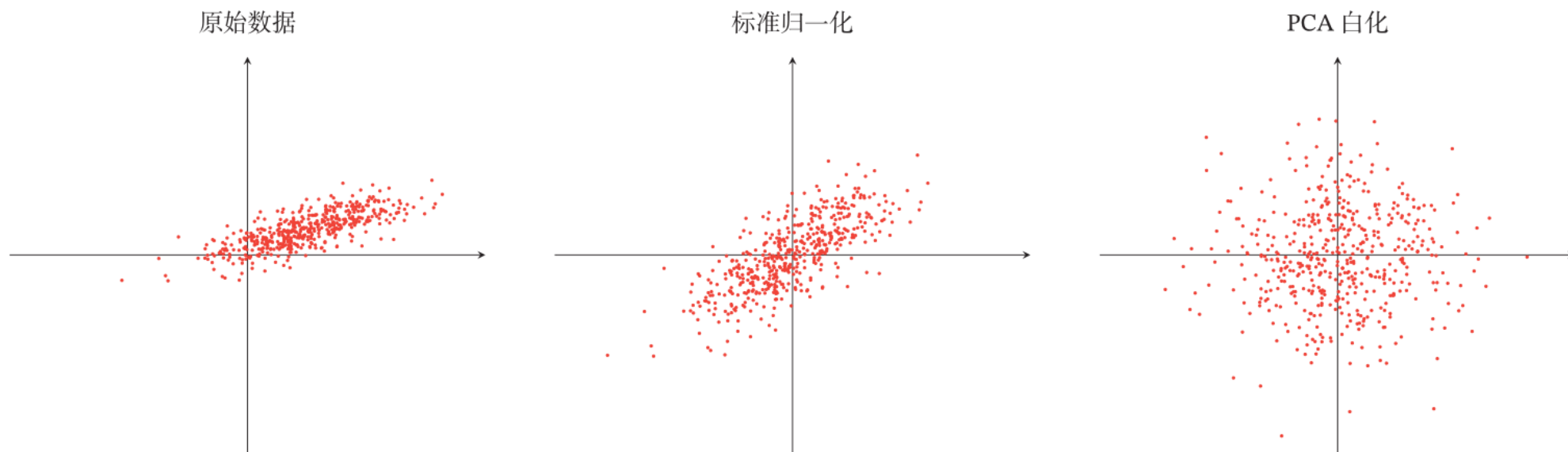
◆ 标准化

◆ 主成分分析 (PCA)

$$\hat{x}^{(n)} = \frac{x^{(n)} - \min_n(x^{(n)})}{\max_n(x^{(n)}) - \min_n(x^{(n)})} \quad [0, 1]$$

$$\hat{x}^{(n)} = \frac{x^{(n)} - \mu}{\sigma} \quad \text{每一个维特征都调整为均值为0, 方差为1}$$

去除掉各个成分之间的相关性



逐层归一化

◆ 目的

◆ 更好的尺度不变性

◆ 内部协变量偏移

◆ 更平滑的优化地形

◆ 归一化方法

◆ 批量归一化 (Batch Normalization, BN)

◆ 层归一化 (Layer Normalization)

◆ 权重归一化 (Weight Normalization)

◆ 局部响应归一化 (Local Response Normalization, LRN)

◆ 批量归一化 (Batch Normalization, BN)

对神经网络中任意的中间层的单个神经元通过一个Batch数据进行标准化

◆ 对于一个深度神经网络, 令第 l 层的净输入为 $\mathbf{z}^{(l)}$, 神经元的输出为 $\mathbf{a}^{(l)}$, 即

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) = f(\mathbf{W}\mathbf{a}^{(l-1)} + \mathbf{b}),$$

◆ 给定一个包含 K 个样本的小批量样本集合的计算均值和方差并

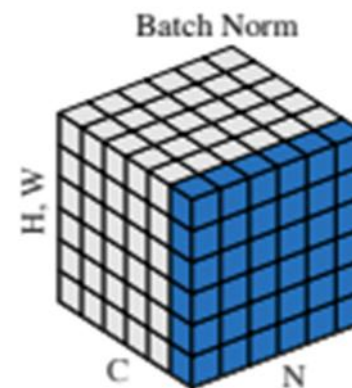
$$\mu_{\mathcal{B}} = \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,l)},$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}) \odot (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}).$$

◆ 归一化

$$\begin{aligned} \hat{\mathbf{z}}^{(l)} &= \frac{\mathbf{z}^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \odot \gamma + \beta \\ &\triangleq \text{BN}_{\gamma, \beta}(\mathbf{z}^{(l)}), \end{aligned}$$

可学习参数



◆ 批量归一化 (Batch Normalization, BN)

◆ 优点

- ◆ 提高优化效率

- ◆ 隐形的正则化方法

- ◆ 在训练时，神经网络对一个样本的预测不仅和该**样本自身**相关，也和同一批次中的**其他样本**相关。由于随机采样，使得神经网络不会“过拟合”到特定样本，从而提高网络的泛化能力。

◆ 缺点

- ◆ 小批量样本的数量不能太小

- ◆ 无法应用到循环神经网络（神经元状态随时间变化）

◆ 层归一化 (Layer Normalization)

对一个中间层的所有神经元进行归一化

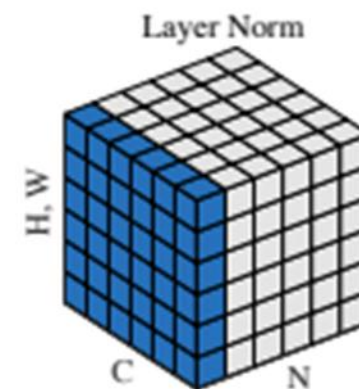
◆ 第 l 层神经元的净输入为 $z^{(l)}$

$$\mu^{(l)} = \frac{1}{n^l} \sum_{i=1}^{n^l} z_i^{(l)},$$

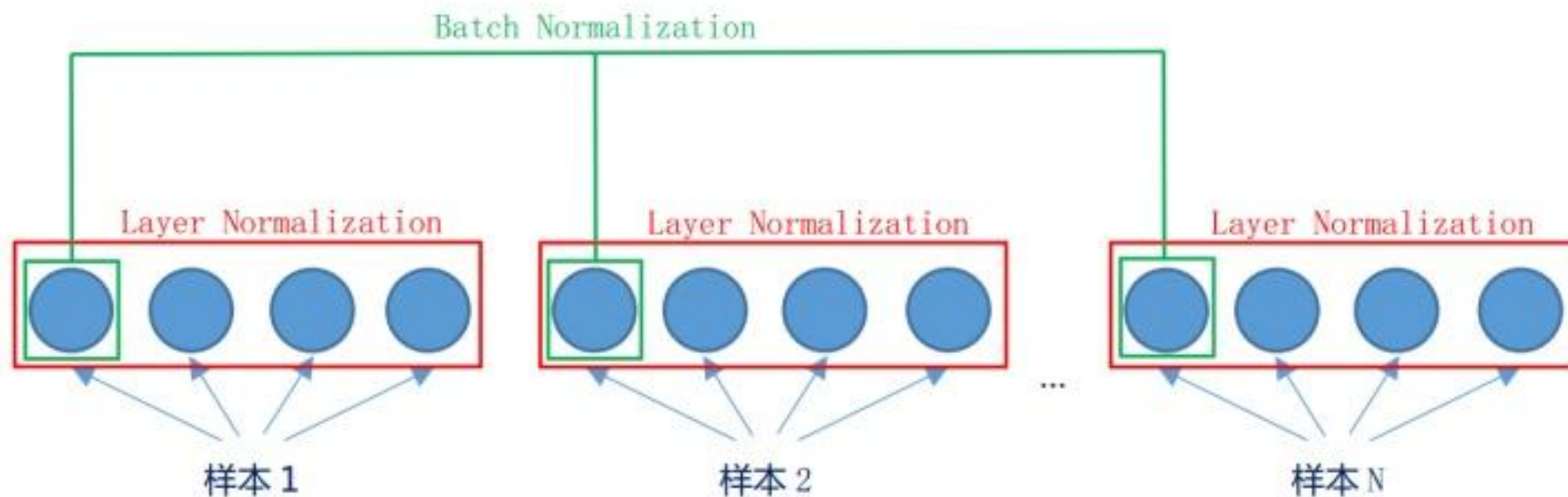
$$\sigma^{(l)2} = \frac{1}{n^l} \sum_{i=1}^{n^l} (z_i^{(l)} - \mu^{(l)})^2,$$

◆ 归一化

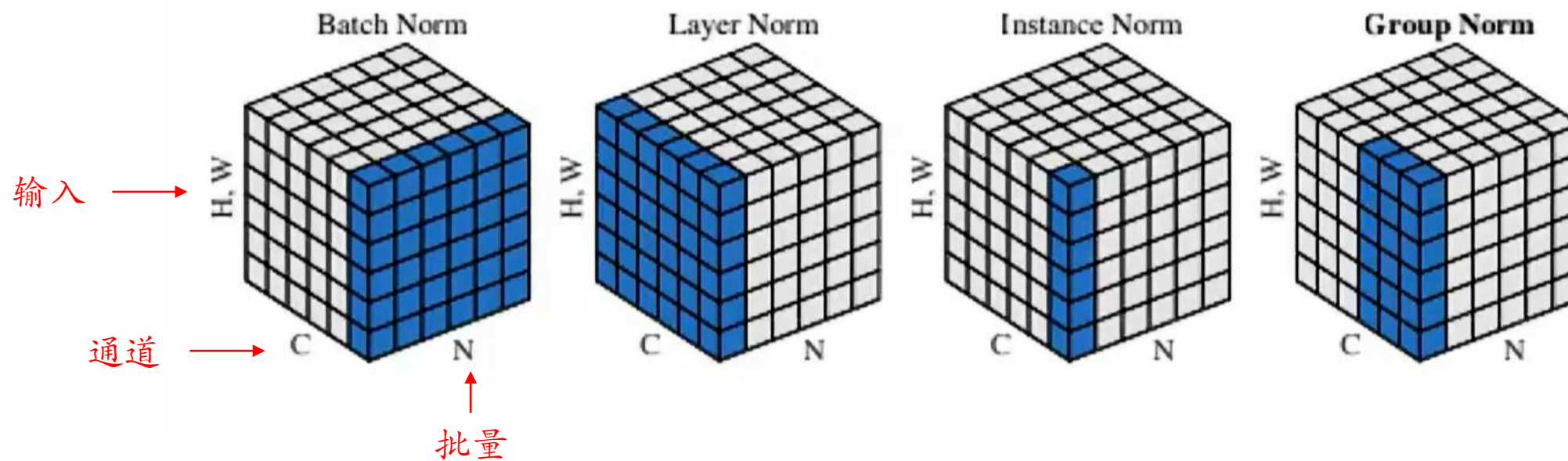
$$\begin{aligned} \hat{z}^{(l)} &= \frac{z^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{(l)2} + \epsilon}} \odot \gamma + \beta \\ &\triangleq \text{LN}_{\gamma, \beta}(z^{(l)}), \end{aligned}$$



◆ 层归一化与批量归一化



其他的归一化



超参数优化

◆ 超参数

◆ 层数

◆ 每层神经元个数

◆ 激活函数

◆ 学习率（以及动态调整算法）

◆ 正则化系数

◆ mini-batch 大小

◆ 优化方法

◆ 网格搜索

◆ 随机搜索

◆ 贝叶斯优化

◆ 动态资源分配

◆ 神经架构搜索

◆ 网格搜索 (Grid Search)

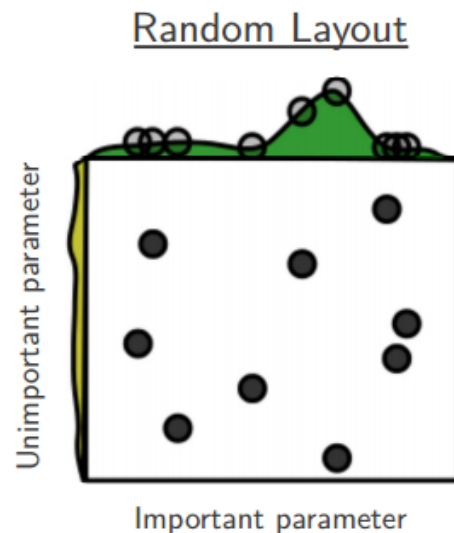
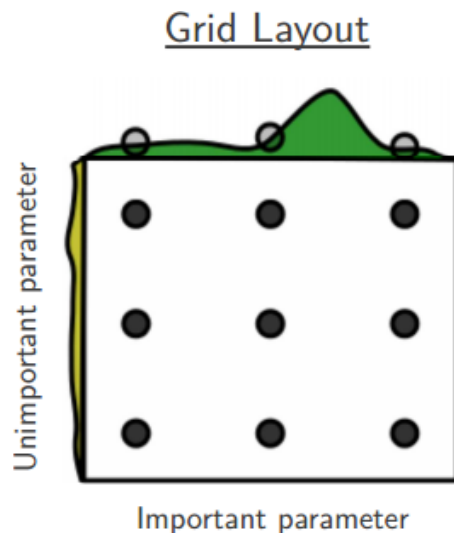
◆ 尝试所有超参数的组合

◆ 假设总共有 K 个超参数，第 k 个超参数可以取 m_k 个值。

◆ 这些超参数可以有 $m_1 \times m_2 \times \cdots \times m_K$ 个取值组合。

◆ 如果参数是连续的，需要根据“经验”离散化。

◆ 随机搜索



每个维度的采样
次数不同

Grid and random search of nine trials for optimizing a function $f(x,y) = g(x) + h(y) \approx g(x)$ with low effective dimensionality. Above each square $g(x)$ is shown in green, and left of each square $h(y)$ is shown in yellow. With grid search, nine trials only test $g(x)$ in three distinct places. With random search, all nine trials explore distinct values of g . This failure of grid search is the rule rather than the exception in high dimensional hyper-parameter optimization.

网络正则化

◆神经网络

◆拟合能力强

◆泛化能力差

在样本真实分布上的期望风险最小化，而训练数据集上的经验风险最小化和期望风险并不一致。



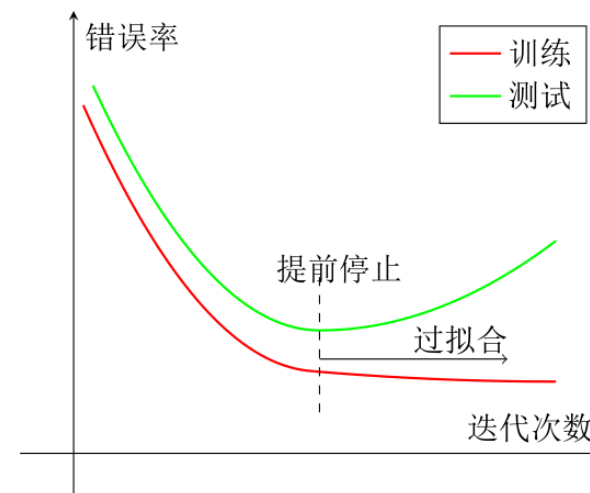
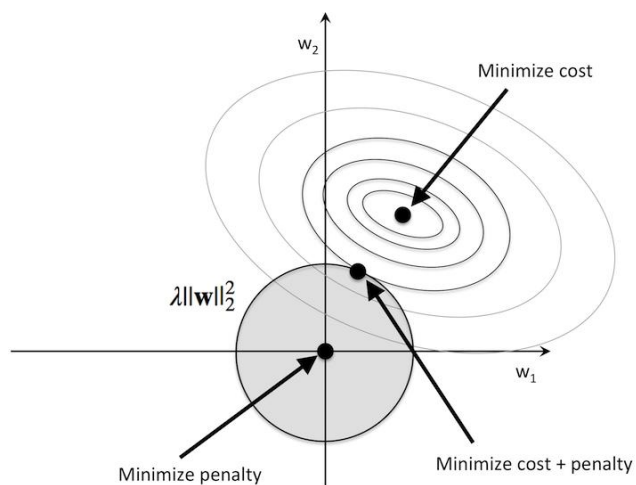
所有损害优化的方法都是正则化。

增加优化约束

干扰优化过程

L1/L2约束、数据增强

权重衰减、随机梯度下降、提前停止、丢弃法

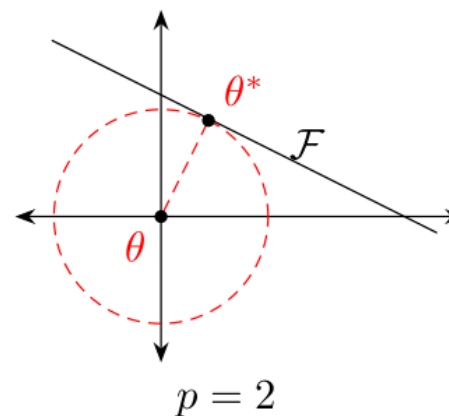
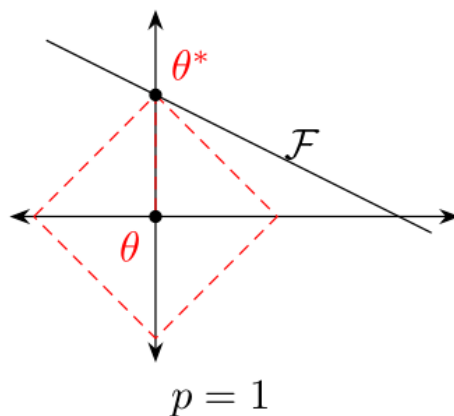
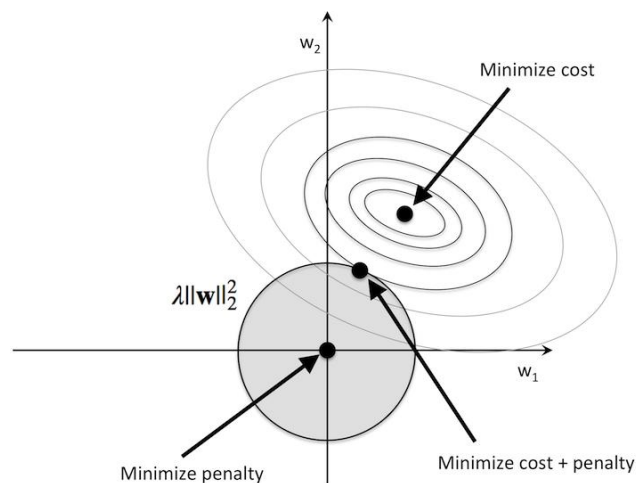


◆ L_p 正则化的优化问题

$$\begin{aligned} \theta^* &= \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \theta)) \\ \text{s.t.} \quad &\ell_p(\theta) \leq 1 \end{aligned}$$

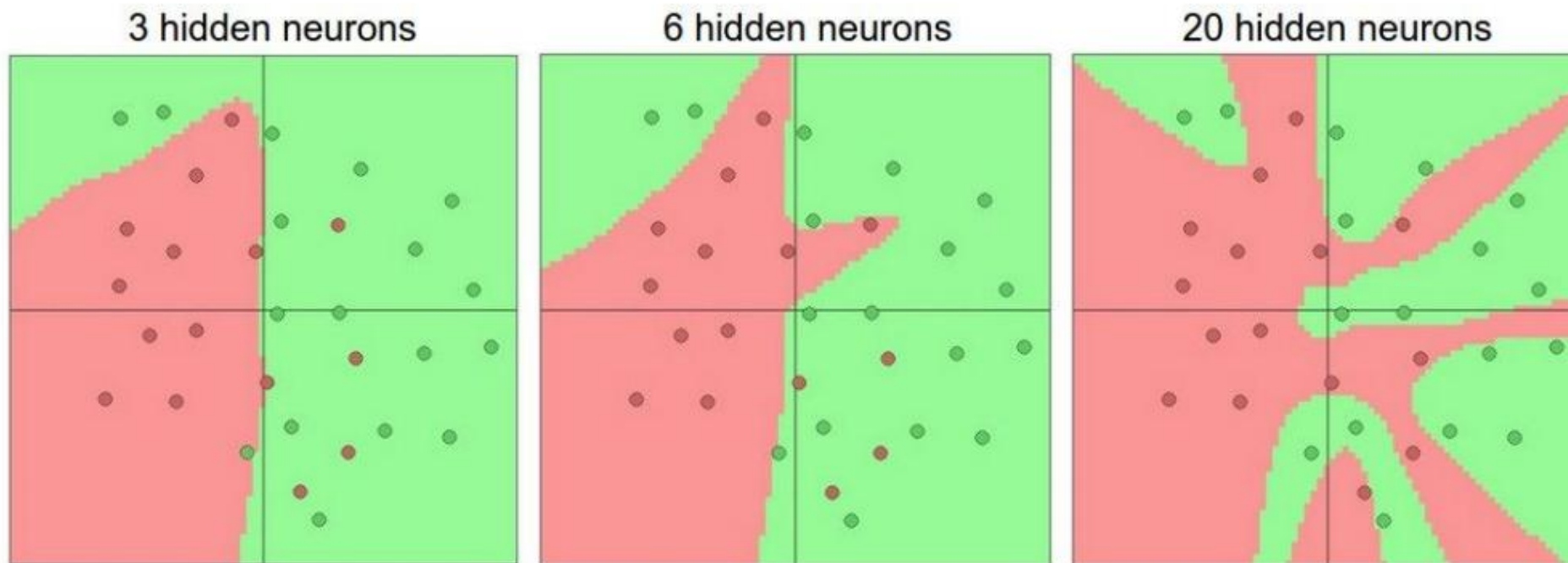
$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, \theta)) + \lambda \ell_p(\theta)$$

ℓ_p 为范数函数, p 的取值通常为 $\{1, 2\}$
代表 ℓ_1 和 ℓ_2 范数, λ 为正则化系数。

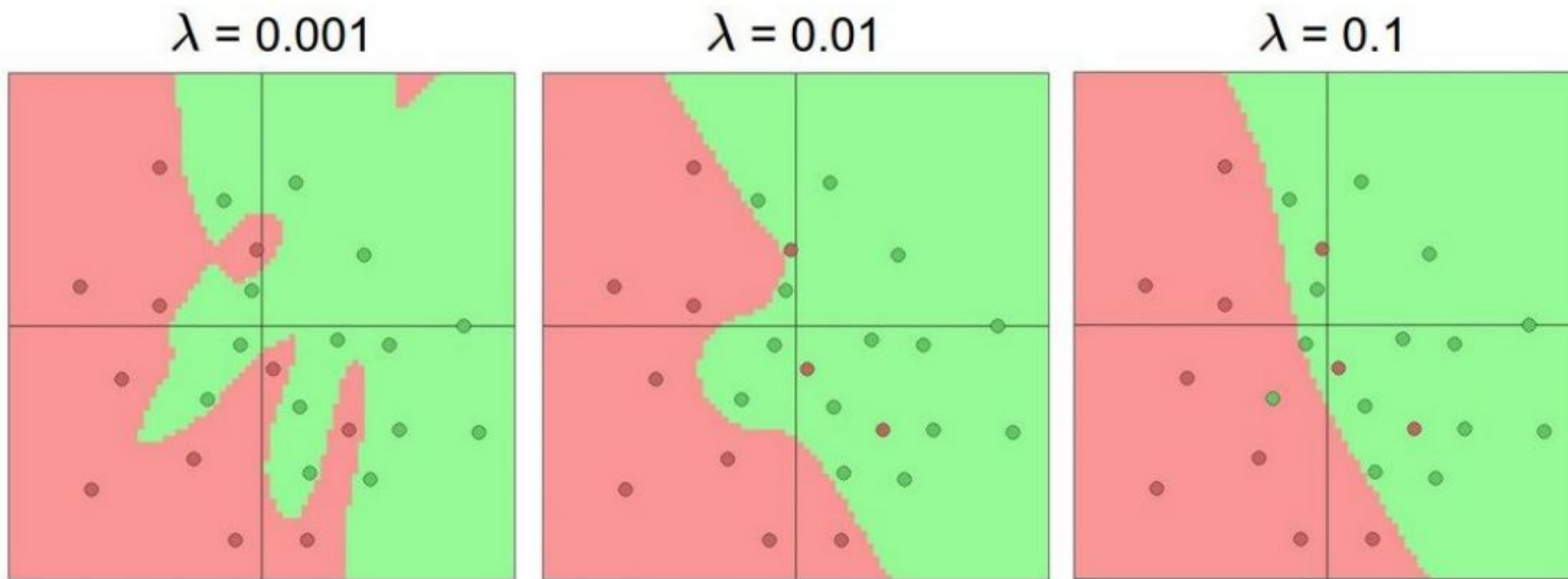


◆ 隐藏层使用不同神经元个数

过拟合



◆使用不同的正则化系数



- ◆通过限制权重的取值范围来干扰优化过程，降低模型能力。
- ◆在每次参数更新时，引入一个衰减系数 β ，一般取值比较小。

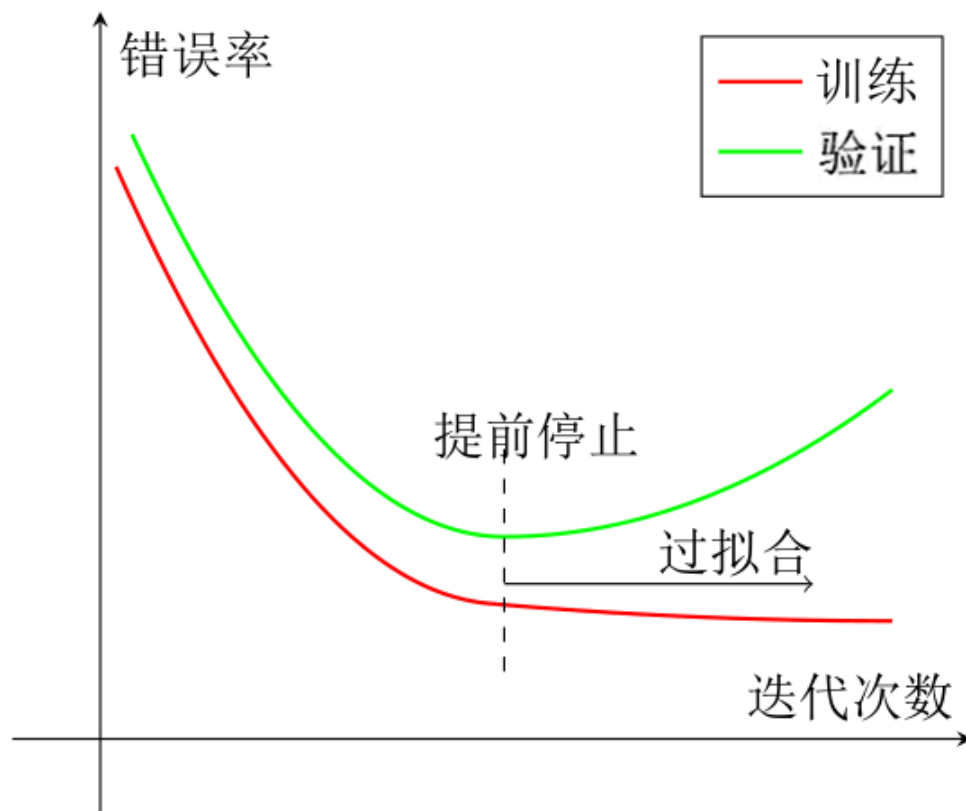
$$\theta_t \leftarrow (1 - \beta)\theta_{t-1} - \alpha \mathbf{g}_t$$

比如 0.0005

- ◆在标准的随机梯度下降中，权重衰减正则化和 ℓ_2 正则化的效果相同；在较为复杂的优化方法（比如Adam）中，权重衰减和 ℓ_2 正则化并不等价。

提前停止 (Early Stop)

- ◆ 使用一个验证集 (Validation Dataset) 来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。



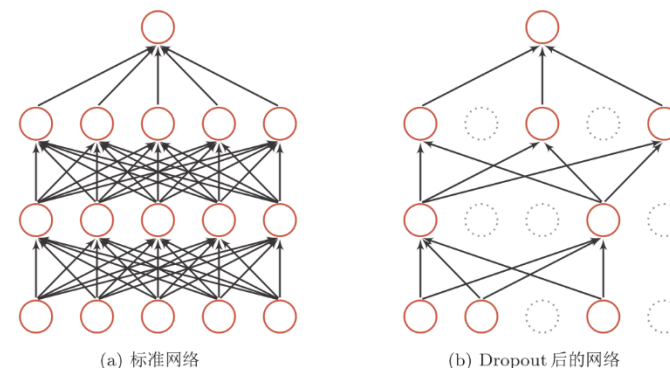
◆ 对于一个神经层，引入一个掩蔽函数使得我们可以随机丢弃一部分神经元（同时丢弃其对应的连接边）来避免过拟合。

$$\text{mask}(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$

◆ 其中 $\mathbf{m} \in \{0,1\}^d$ 是丢弃掩码 (dropout mask)，通过以概率为 p 的伯努利分布随机生成。

◆ 丢弃法可以用集成学习和贝叶斯学习来解释。

训练和测试时网络
的输出不一致



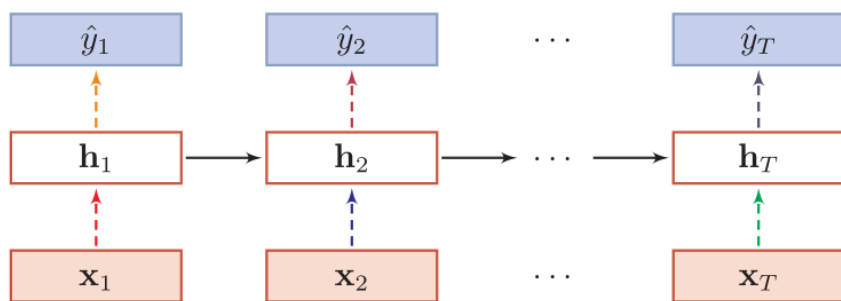
◆ 当在循环神经网络上应用丢弃法，不能直接对每个时刻的隐状态进行随机丢弃，这样会损害循环网络在时间维度上记忆能力。

◆ 变分Dropout

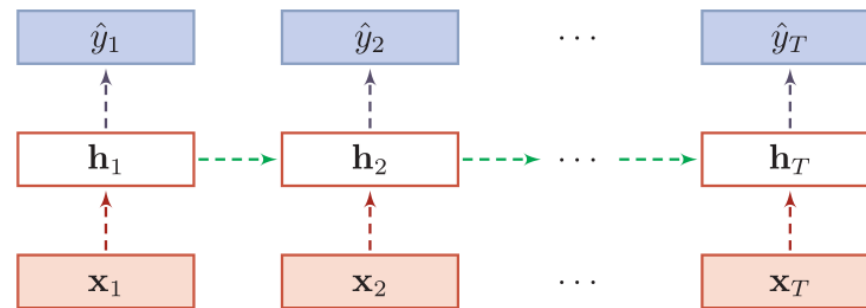
◆ 根据贝叶斯学习的解释，丢弃法是一种对参数 θ 的采样。

◆ 每次采样的参数需要在每个时刻保持不变。

◆ 在对循环神经网络上使用丢弃法时，需要对参数矩阵的每个元素进行随机丢弃，并在所有时刻都使用相同的丢弃掩码。



虚线边表示进行随机丢弃，不同的颜色表示不同的丢弃掩码。



相同颜色表示使用相同的丢弃掩码。

◆ 图像数据的增强主要是通过算法对图像进行转变，引入噪声等方法来增加数据的多样性。

◆ 图像数据的增强方法：

- ◆ 旋转 (Rotation)：将图像按顺时针或逆时针方向随机旋转一定角度；
- ◆ 翻转 (Flip)：将图像沿水平或垂直方法随机翻转一定角度；
- ◆ 缩放 (Zoom In/Out)：将图像放大或缩小一定比例；
- ◆ 平移 (Shift)：将图像沿水平或垂直方法平移一定步长；
- ◆ 加噪声 (Noise)：加入随机噪声。

◆ 文本数据的增强方法：

- ◆ 同义词替换
- ◆ 回译 (Back Translation)
- ◆ 随机编辑噪声
 - ◆ 增删改查
 - ◆ 句子乱序

- ◆ 在输出标签中添加噪声来避免模型过拟合。
- ◆ 一个样本的标签一般用one-hot向量表示，这是一种**硬目标**

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]^T$$

- ◆ 引入一个噪声对标签进行平滑，即假设样本以 ϵ 的概率为其它类。平滑后的标签为

$$\tilde{\mathbf{y}} = [\frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}, 1 - \epsilon, \frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}]^T$$

软目标

- ◆ **截止时间：** 下周日晚9:00
- ◆ **提交方式：** 每人作业以 pdf 格式提交，命名为“学号_姓名”，班长收齐将全班作业压缩包发送到邮箱：nuoxu25@mails.jlu.edu.cn(许诺)
- ◆ **作业晚交政策：** 作业超过截止时间但在24小时以内，该次作业按照60%分数计算，超过24小时不计分

习题 7-1 在小批量梯度下降中,试分析为什么学习率要和批量大小成正比.

习题 7-2 在Adam算法中,说明指数加权平均的偏差修正的合理性(即公式(7.27)和公式(7.28)).

$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t}, \quad (7.27)$$

$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}. \quad (7.28)$$